MarketMind AI

Small Business Sales Intelligence Platform

Milestone 1 Documentation

(Week 1 & Week 2)

Infosys Springboard Virtual Internship 7.0

Team 1

Mentor:

Shravya

Team Members:

Akshaya Bale

Tejananda

Divyanka Singh

Garvit

Komal Kumari

Vishnu Vardhan

Hari Vimal

Submission Date - 30 July 2026

Contents

1. Project Overview

1.1 Introduction

MarketMind AI is an AI-enabled Small Business Sales Intelligence Platform developed to help retail businesses efficiently manage sales, inventory, customers, and business operations through a centralized web application. The platform provides role-based access for different users within an organization and enables secure management of operational data through a modern web interface.

The project combines modern web technologies with business intelligence concepts to provide meaningful insights that support business decision-making. The application is designed to integrate frontend, backend, and database components into a single platform while maintaining secure authentication and authorization mechanisms.

During Milestone 1, the primary focus was to establish the project foundation by collecting and preprocessing retail datasets, designing the overall system architecture and database, initializing the frontend and backend, implementing authentication and Role-Based Access Control (RBAC), and developing role-specific dashboards.

1.2 Problem Statement

Small businesses often rely on multiple disconnected tools for managing sales transactions, inventory, customer information, and reporting. This leads to duplicated data, inefficient workflows, limited visibility into business performance, and increased operational complexity.

The objective of MarketMind AI is to provide a unified platform where business owners and employees can securely access business information based on their responsibilities. By combining centralized data management with AI-powered analytics, the system aims to improve operational efficiency and support data-driven decision making.

1.3 Project Objectives

The primary objectives of MarketMind AI are:

- Develop a centralized retail sales management platform.
- Provide secure authentication and role-based access control.
- Design a scalable database architecture for retail business operations.
- Build responsive dashboards for multiple user roles.
- Collect and preprocess retail datasets for future AI model development.
- Integrate frontend, backend, and database components.
- Establish a reusable project architecture for future milestones.
- Prepare the application for advanced analytics and machine learning features.

1.4 Scope of Milestone 1

Milestone 1 focuses on establishing the core foundation of the MarketMind AI platform. The work completed during this milestone includes:

- Collection of retail sales and inventory datasets.
- Analysis of dataset structure and business attributes.
- Data cleaning and preprocessing.
- Definition of project objectives and business workflow.
- Identification of user roles and system responsibilities.
- Design of the overall system architecture.
- Design of the relational database schema.
- UI wireframe creation and dashboard planning.
- Frontend initialization using React.js and Vite.
- Backend initialization using FastAPI.
- Implementation of authentication and Role-Based Access Control (RBAC).
- Development of role-based dashboards.
- Initial integration between frontend, backend, and database.
- GitHub repository setup and collaborative branch-based development.

The deliverables produced in this milestone establish the technical and functional foundation required for the AI modules and advanced business intelligence features planned for future milestones.

2. Milestone Objectives

The following table maps the objectives assigned for Milestone 1 with the completion status achieved by the team.

Milestone 1 Completion Status

Milestone Requirement	Status	Evidence
Collect retail sales and inventory datasets	✅ Completed	Retail sales, inventory, pricing, training and testing datasets collected and organized

Milestone Requirement	**Status**	**Evidence**
Analyze dataset structure and business attributes	✅ Completed	Dataset analysis, EDA, business attribute analysis, and preprocessing completed
Define Project Statement, Objectives & Workflow	✅ Completed	Project Statement, Objectives, Business Workflow, and User Roles document prepared
Design System Architecture	✅ Completed	System Architecture diagram and authenticated request flow designed
Design Database Structure	✅ Completed	ER Diagram and relational database schema completed
Create UI Wireframes	✅ Completed	UI wireframes and dashboard layouts designed for all user roles
Dashboard Planning	✅ Completed	Business Owner, Store Manager, Sales Executive, and System Administrator dashboards planned
Clean & Preprocess Transaction Records	✅ Completed	Data cleaning, preprocessing pipeline, train-test split, validation, and processed datasets prepared
Process Inventory & Customer Datasets	✅ Completed	Inventory and customer datasets processed and validated
Build Initial Analytics Dashboard	✅ Completed	Role-based dashboards implemented using React
Initialize Backend using FastAPI	✅ Completed	FastAPI project structure with APIs, models and services implemented
Initialize Frontend using React + Vite	✅ Completed	React + Vite application initialized with routing and reusable components
Implement Authentication	✅ Completed	Login, JWT authentication and protected routes implemented

Milestone Requirement	Status	Evidence
Implement Role-Based Access Control (RBAC)	✅ Completed	Role-based permissions implemented for four user roles
Implement User Permissions	✅ Completed	Permission matrix and authorization rules configured
Implement Dashboard Access Management	✅ Completed	Separate dashboards for each user role
Frontend–Backend Integration	✅ Completed	Frontend connected with backend APIs and database
GitHub Repository & Branch Management	✅ Completed	Feature branch workflow and Milestone 1 integration branch used
Project Documentation	✅ Completed	Milestone documentation under preparation

3. Team Members & Responsibilities

Team Member	Contribution
Akshaya Bale	Project Statement, Objectives, User Roles, Workflow, System Architecture, Database Design, Documentation
Tejananda	Dashboard UI, Components, React Frontend
Divyanka Singh	React Setup, Authentication UI, Frontend Integration
Garvit	Backend Development, APIs, Authentication, RBAC, FastAPI
Komal	Dataset Collection, Cleaning, Preprocessing, EDA
Vishnu Vardhan	Additional Frontend Components & Admin UI
Hari Vimal	-

4. Work Completed

4.1 Dataset Collection

Overview

The first phase of the project focused on collecting suitable datasets required for building a Small Business Sales Intelligence Platform. Since the objective of MarketMind AI is to provide business insights, forecasting, inventory management, and customer analytics, the datasets were selected to represent real-world retail business operations.

The collected datasets include transaction records, inventory information, pricing details, and supporting data required for model training and system development. These datasets serve as the foundation for future AI-based analytics and decision-making modules.

Datasets Collected

Dataset	Description	Purpose
Retail Sales Dataset	Sales transactions including products, quantity, revenue, and dates	Sales analysis and reporting
Inventory Dataset	Product inventory and stock information	Inventory management
Pricing Dataset	Product pricing and category details	Revenue calculations
Training & Testing Datasets	Processed datasets prepared for analytics and future AI models	Machine learning preparation

Outcome

The dataset collection phase successfully identified and organized all required datasets. These datasets were stored in a structured format and used for analysis, preprocessing, and application development.

4.2 Dataset Analysis

Overview

After collecting the datasets, a detailed analysis was performed to understand their structure, attributes, and business relevance. This step helped identify the relationships between different datasets and determine how they would be integrated into the application.The analysis focused on understanding the available business information, identifying important features, and preparing the data for preprocessing.

Analysis Performed

The following activities were carried out during dataset analysis:

- Examination of dataset structure.
- Identification of important business attributes.
- Review of sales-related fields.
- Analysis of inventory records.

- Identification of missing values.
- Identification of duplicate records.
- Verification of data types.
- Validation of relationships between business entities.

Business Attributes Identified

The datasets contain information related to:

- Product Details
- Sales Transactions
- Customer Information
- Inventory Records
- Product Categories
- Revenue
- Quantity Sold
- Transaction Date
- Store Information

These attributes provide the necessary information for generating dashboards, reports, and future AI-based insights.

Outcome

The analysis confirmed that the collected datasets contained sufficient information to support business intelligence features. It also identified preprocessing requirements such as handling missing values, duplicate records, and inconsistent formatting.

4.3 Data Cleaning & Preprocessing

Overview

Raw datasets often contain inconsistencies that can negatively affect application performance and future machine learning models. Therefore, a comprehensive data preprocessing pipeline was implemented before integrating the data into the system.

The preprocessing process focused on improving data quality, ensuring consistency, and preparing datasets for database storage and analytics.

Data Cleaning Activities

The following preprocessing tasks were completed:

- Removal of duplicate records.
- Handling missing values.
- Correction of inconsistent data formats.
- Standardization of categorical values.
- Date format normalization.
- Validation of numerical fields.
- Removal of invalid records.
- Data integrity verification.

Tools Used

Tool	Purpose
Python	Data preprocessing
Pandas	Data manipulation
NumPy	Numerical operations
Scikit-learn	Dataset preparation
CSV Files	Dataset storage

Results

The preprocessing stage produced clean and validated datasets that were suitable for:

- Dashboard visualization.
- Database integration.
- API development.
- Business reporting.
- Future AI model training.

The processed datasets were then integrated into the backend and made available for frontend visualization through the FastAPI application.

4.4 Project Objectives & Business Workflow

Business Workflow

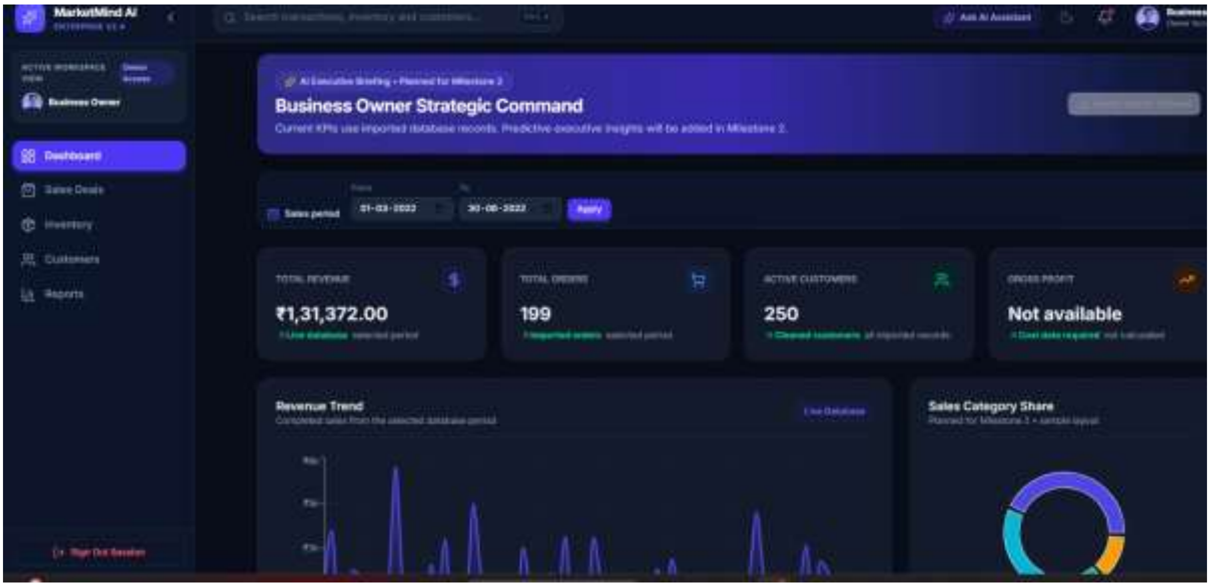
The overall workflow of the application consists of the following steps:

1. Users access the application through the login page.

2. The system authenticates user credentials using JWT Authentication.
3. Based on the assigned role, the system grants appropriate permissions.
4. The user is redirected to the corresponding dashboard.
5. The dashboard retrieves required business data through REST APIs.
6. The backend processes the request and interacts with the PostgreSQL database.
7. The requested information is displayed on the dashboard.
8. Any updates performed by users are stored securely in the database and reflected throughout the application.

This workflow ensures secure communication between all system components while maintaining role-based access to business information.

Figure 4.1: *Business Workflow Diagram*



4.5 User Roles

Role	Responsibility
Business Owner	Business monitoring and strategic decisions
Store Manager	Inventory and daily operations
Sales Executive	Sales and customer management
Administrator	Users, roles, security

User Role Summary

User Role	Primary Responsibility
Business Owner	Business monitoring and strategic decision-making
Store Manager	Inventory and daily operations management
Sales Executive	Sales transactions and customer management
System Administrator	User management, authentication, and system administration

---

4.6 System Architecture

Overview

The MarketMind AI platform follows a modern three-tier architecture that separates the application into frontend, backend, and database layers. This modular design improves scalability, maintainability, and security while allowing each layer to be developed and maintained independently.

The architecture enables secure communication between the user interface and backend services using REST APIs protected by JWT Authentication.

Architecture Components

Frontend Layer

The presentation layer is developed using **React.js** with **Vite**. It provides responsive dashboards, navigation, authentication interfaces, and role-specific modules.

- Navigation
- API Calls
- Dashboard

Backend Layer

The backend is developed using **FastAPI** and exposes RESTful APIs for frontend communication.

- REST APIs
- Authentication
- Business Logic

Database Layer

The application uses **PostgreSQL** as the relational database management system.

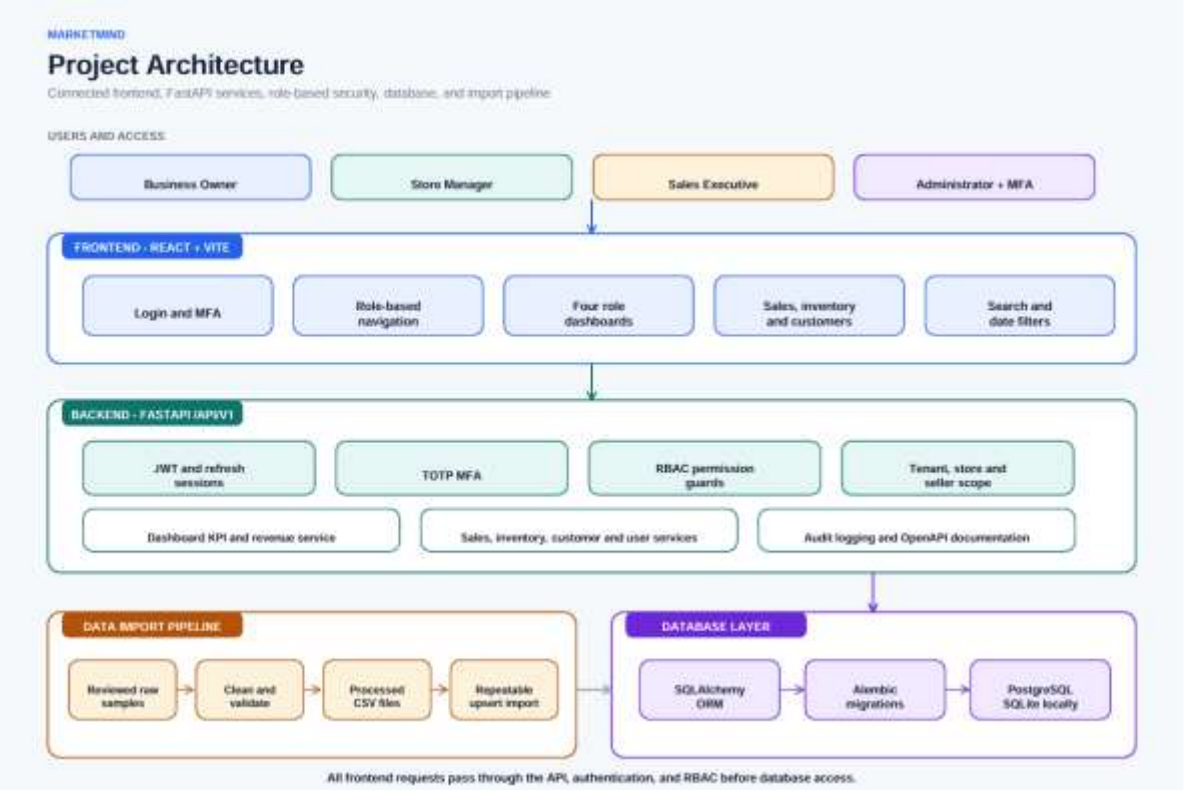
- PostgreSQL Storage

Request Flow

The communication between components follows these steps:

1. User submits a request through the React frontend.
2. Axios sends an HTTP request to the FastAPI backend.
3. JWT token is verified.
4. RBAC permissions are validated.
5. Backend executes business logic.
6. SQLAlchemy interacts with PostgreSQL.
7. Database returns the requested data.
8. Backend sends the API response.
9. React updates the dashboard.

Figure 4.2: *System Architecture Diagram*



4.7 Database Design

Overview

The database was designed to efficiently store and manage business information while maintaining relationships between different business entities. A relational database model was selected to ensure data consistency, integrity, and scalability.

The schema supports authentication, sales management, inventory tracking, customer records, and role-based user management.

Main Database Entities

The database consists of the following major entities:

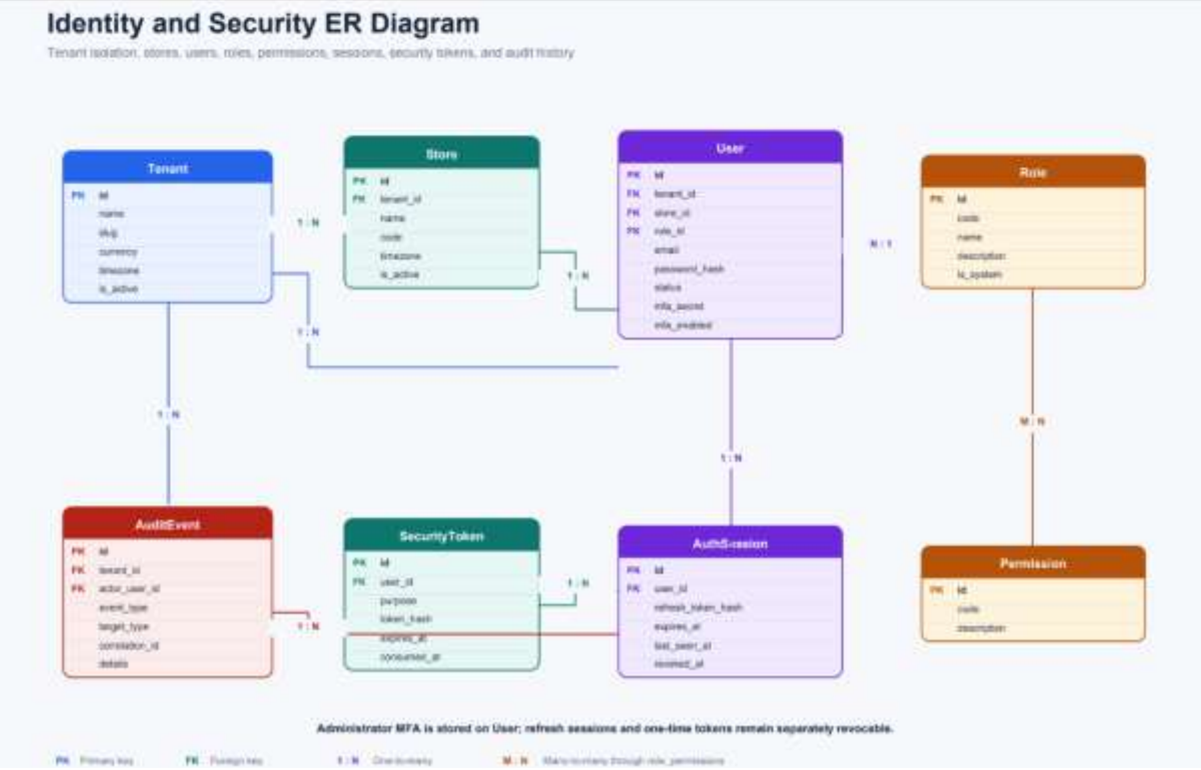
- Users
- Roles
- Products
- Categories
- Inventory
- Customers
- Sales
- Sales Items
- Reports

These entities are interconnected through primary and foreign key relationships, enabling efficient querying and data management.

Entity Relationship Diagram (ER Diagram)

The Entity Relationship Diagram (ERD) illustrates the relationships between all major entities used within the application. It serves as the blueprint for database implementation and ensures consistency between application modules.

Figure 4.3: *Entity Relationship Diagram (ERD)*



Database Integration

The PostgreSQL database is integrated with the FastAPI backend using SQLAlchemy ORM. Database operations such as Create, Read, Update, and Delete (CRUD) are performed through service and repository layers, ensuring clean separation of concerns and maintainable code.

4.8 UI Wireframes & Dashboard Planning

Overview

Before beginning frontend development, the team designed the user interface to ensure a consistent, intuitive, and responsive user experience. The wireframes served as the initial blueprint for developing the application and helped define the placement of navigation menus, dashboard widgets, data tables, charts, and forms.

The dashboard layouts were planned according to the responsibilities of each user role so that users would only have access to information relevant to their tasks. This planning phase reduced development complexity and ensured consistency across all modules.

Dashboard Design Approach

The following design principles were considered during dashboard planning:

- Simple and user-friendly interface.
- Responsive layout for different screen sizes.
- Easy navigation between modules.
- Clear visualization of business metrics.
- Reusable UI components.
- Role-specific information display.
- Consistent color scheme and typography.

Dashboard Modules

The application dashboard consists of the following modules:

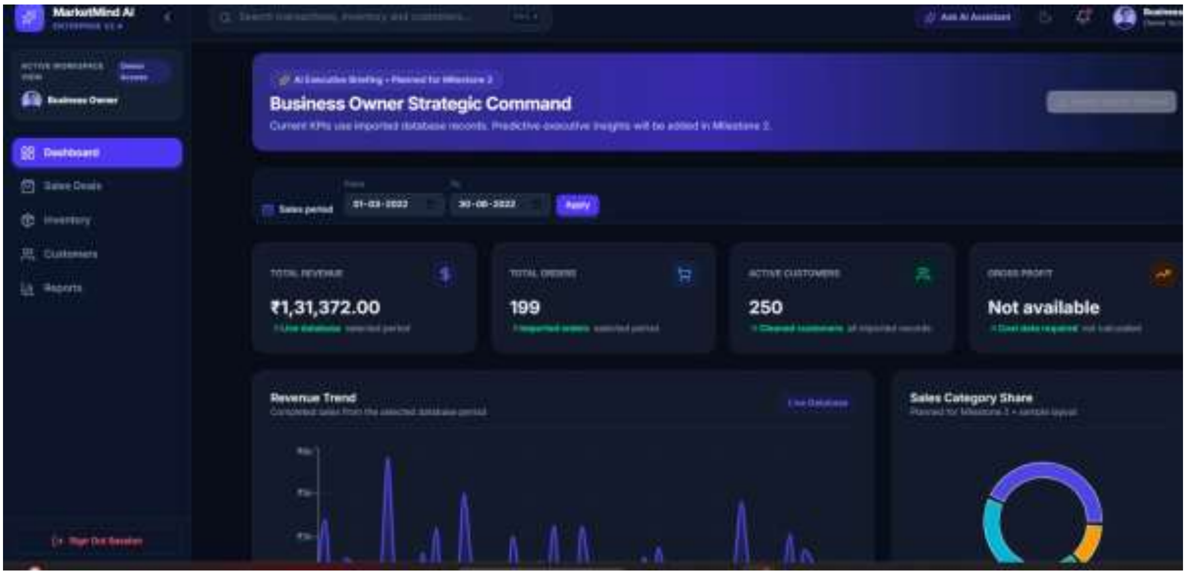
- Dashboard
- Sales Management
- Inventory Management
- Customer Management
- Reports
- Settings
- User Profile

Each module is accessible according to the user's assigned role.

Planned Dashboards

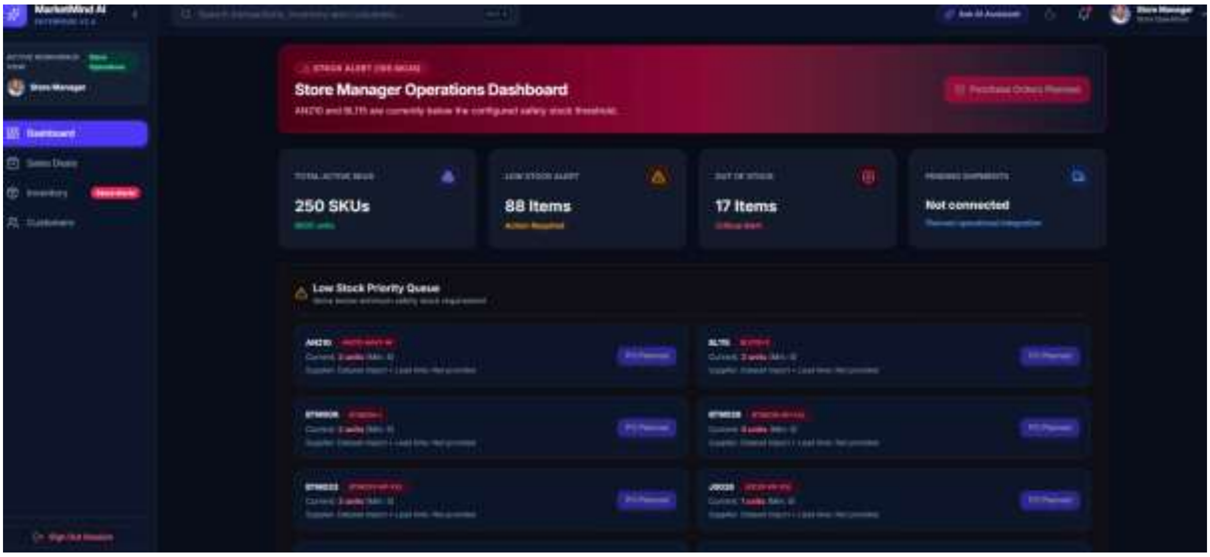
Four dedicated dashboards were designed and implemented:

Figure 4.4: *Business Owner Dashboard Screenshot*



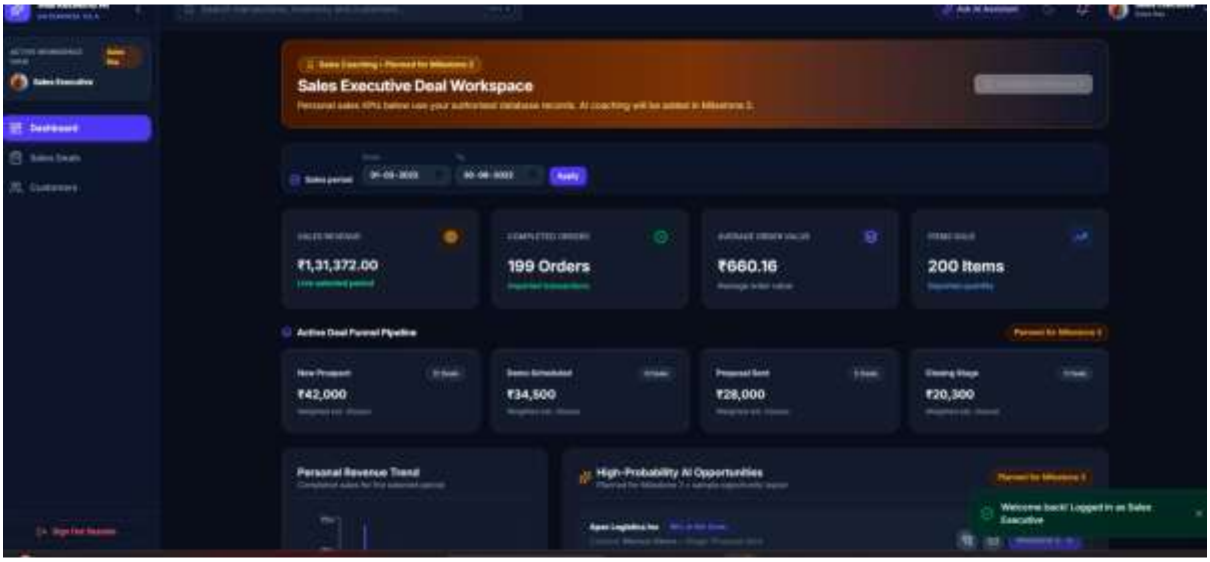
The Business Owner Dashboard provides an overview of the organization's performance through key business metrics and analytical reports.

Figure 4.5: *Store Manager Dashboard Screenshot*



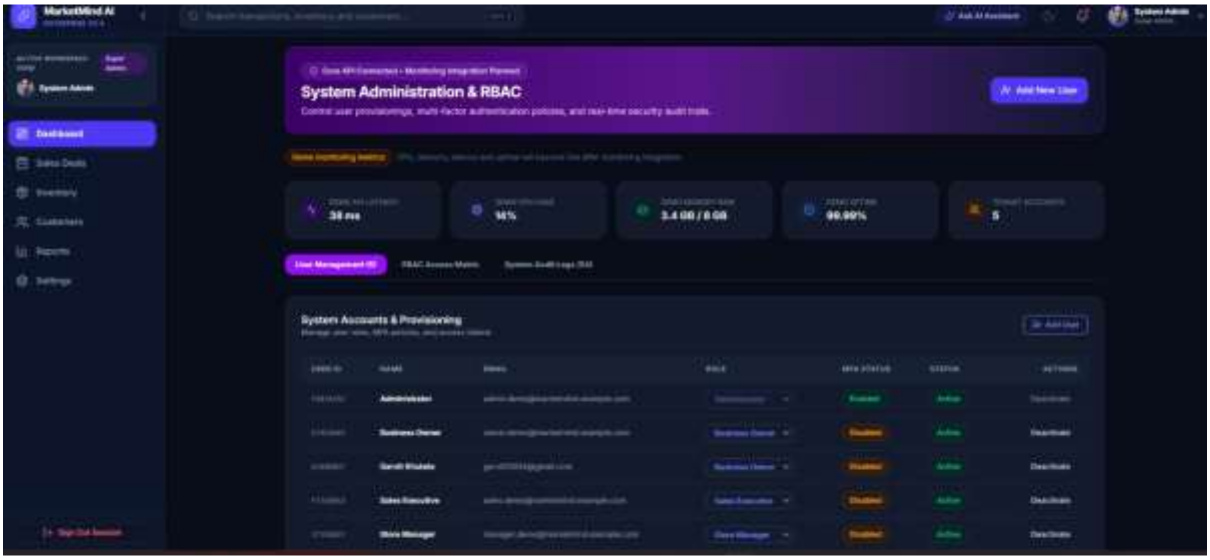
The Store Manager Dashboard focuses on daily operational activities.

Figure 4.6: *Sales Executive Dashboard Screenshot*



The Sales Executive Dashboard is designed to simplify customer transactions and sales activities.

Figure 4.7: *System Administrator Dashboard Screenshot*



The System Administrator Dashboard provides tools for managing users and system configurations.

Outcome: The UI planning phase ensured that each dashboard was tailored to the responsibilities of its respective user role. The finalized layouts served as the foundation for frontend development and simplified the integration of backend APIs.

4.9 Frontend Development

Overview

The frontend of MarketMind AI was developed using **React.js** and **Vite**, providing a modern, fast, and responsive user interface. The application was designed using a component-based architecture, enabling reusable UI components and easier maintenance.

The frontend communicates with the FastAPI backend through REST APIs to retrieve and display application data. It also manages user authentication, dashboard rendering, navigation, and role-based access.

Technologies Used

Technology	Purpose
React.js	User interface development
Vite	Build tool and development server
JavaScript (ES6+)	Application logic
React Router	Navigation and routing
Axios	API communication
Context API	Global state management
CSS	Application styling
Lucide React	Icons

Frontend Architecture

The application follows a modular architecture where functionality is divided into reusable components.

Major folders include:

- components/
- pages/
- context/
- services/
- assets/
- styles/

This structure improves maintainability, scalability, and code reuse.

Figure 4.8: *Frontend Project Structure*

```
frontend/
|── public/
|── src/
|   ├── assets/
|   ├── components/
|   |   ├── auth/
|   |   ├── common/
|   |   ├── dashboards/
|   |   ├── modules/
|   |   └── ui/
|   |
|   ├── context/
|   ├── data/
|   ├── services/
|   ├── App.jsx
|   ├── main.jsx
|   └── index.css
|
├── package.json
├── vite.config.js
└── README.md
```

State Management

Global application state is managed using React Context API.

The following contexts were implemented:

- AuthContext
- ThemeContext
- ToastContext

These contexts handle authentication, application themes, and notification messages across different components.

API Integration

Axios is used to communicate with the FastAPI backend.

The frontend consumes APIs for:

- User authentication

- Dashboard data
- User management
- Sales management
- Inventory management
- Customer management
- Reports
- System administration

Outcome

The frontend successfully provides an intuitive and responsive interface for different user roles while ensuring secure communication with backend services.

4.10 Backend Development

Overview

The backend of MarketMind AI was developed using **FastAPI**, providing high-performance REST APIs for frontend communication. The backend follows a modular architecture that separates API routes, business logic, database operations, authentication, and configuration into independent modules.

This architecture improves scalability, maintainability, and ease of future development.

Technologies Used

Technology	Purpose
FastAPI	REST API framework
PostgreSQL	Relational database
SQLAlchemy	ORM
Pydantic	Data validation
JWT	Authentication
Passlib	Password hashing
Alembic	Database migrations

Technology	Purpose
Docker	Containerization

Backend Modules

The backend consists of the following modules:

- API Routes
- Authentication
- Authorization
- Database Models
- Schemas
- Services
- Database Configuration
- Utility Functions

Figure 4.9: *Backend Project Structure*

```
backend/
│
├── alembic/
├── app/
│   ├── api/
│   ├── commands/
│   ├── core/
│   ├── db/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   └── main.py
│
├── data/
│   ├── raw/
│   └── processed/
│
├── docs/
├── tests/
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml
└── README.md
```

Backend Features

Implemented features include:

- RESTful API development.
- JWT Authentication.
- Role-Based Access Control.
- CRUD operations.
- User management.
- Sales management.
- Inventory management.
- Customer management.
- Database integration.
- Error handling.

API Development

Developed REST APIs for authentication, dashboard, sales, inventory, customer and reporting modules.

Database Connectivity

The backend interacts with PostgreSQL using SQLAlchemy ORM, ensuring efficient CRUD operations while maintaining data integrity and abstraction from raw SQL queries.

Outcome

The backend establishes the core business logic of the application and provides secure, scalable APIs that integrate seamlessly with the frontend and database.

4.11 Authentication & Role-Based Access Control (RBAC)

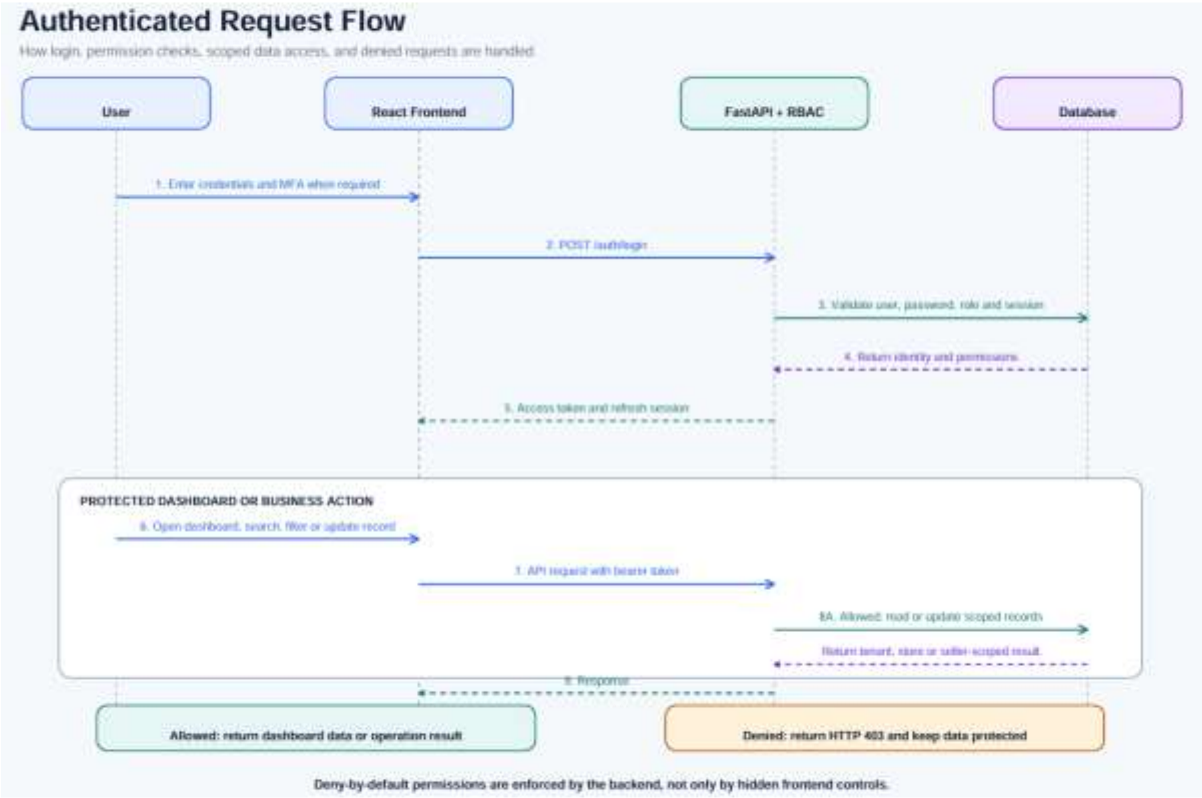
Authentication Workflow

The authentication process follows these steps:

1. User enters login credentials.

2. Credentials are validated by the backend.
3. Passwords are securely verified.
4. JWT access token is generated.
5. Token is returned to the frontend.
6. Token is stored securely for subsequent requests.
7. Protected API requests include the JWT token.
8. Backend validates the token before processing requests.

Figure 4.10: *Authentication Flow Diagram*



Role-Based Access Control

The application implements RBAC to restrict access according to user roles.

Supported roles include:

- Business Owner
- Store Manager

- Sales Executive
- System Administrator

Each role has predefined permissions for accessing modules, performing operations, and managing data.

Permission Matrix

Module	Business Owner	Store Manager	Sales Executive	System Administrator
Dashboard	✓	✓	✓	✓
Sales	✓	✓	✓	View
Inventory	✓	✓	View	View
Customers	✓	✓	✓	View
Reports	✓	✓	View	View
User Management	✕	✕	✕	✓
Role Management	✕	✕	✕	✓
Settings	✓	Limited	✕	✓

4.12 GitHub Repository Structure

Overview

GitHub was used as the primary version control system throughout the development of MarketMind AI. It enabled collaborative development by allowing team members to work independently on different modules while maintaining a centralized code repository. A structured branching strategy was followed to ensure smooth integration and minimize conflicts during development.

Repository Organization

The project repository is organized into separate frontend and backend components along with configuration files and project documentation.

Frontend Repository

The frontend repository contains the React.js application, reusable UI components, pages, routing, API services, and application assets.

Major folders include:

- components/
- pages/
- context/
- services/
- assets/
- styles/

Backend Repository

The backend repository contains the FastAPI application, API routes, database models, schemas, services, authentication modules, and migration scripts.

Major folders include:

- app/
- api/
- models/
- schemas/
- services/
- core/
- db/
- alembic/

Branching Strategy

To support collaborative development, Git feature branches were used.

The workflow followed these steps:

1. Clone the repository.

2. Create a feature branch.
3. Develop the assigned module.
4. Commit changes with descriptive messages.
5. Push changes to GitHub.
6. Create a Pull Request.
7. Review and merge into the integration branch.
8. Test the integrated application.

This workflow ensured code quality and reduced merge conflicts.

Figure 4.11: *GitHub Repository Structure*

```
main
|
├── Akshaya29
├── Garvitk001
├── komal283
├── divyanka-0525
├── Tejananda
├── vishnu
└── milestone-1-integration
```

4.13 Integration Status

Overview

One of the primary objectives of Milestone 1 was to integrate the frontend, backend, and database into a functional application. The integration phase verified that all major components could communicate successfully while maintaining secure authentication and role-based access.

Integration Components

The following components were integrated successfully:

Component	Status
React Frontend	✅ Integrated
FastAPI Backend	✅ Integrated
PostgreSQL Database	✅ Connected
Authentication	✅ Working
JWT Authorization	✅ Working
RBAC	✅ Implemented
REST APIs	✅ Connected
Dashboard Modules	✅ Functional

Integration Workflow

The integration process follows these steps:

1. User logs into the application.
2. React frontend sends credentials to FastAPI.
3. Backend authenticates the user.
4. JWT token is generated.
5. Frontend stores the authentication token.
6. Protected API requests include the JWT token.
7. Backend validates permissions.
8. Database processes the request.
9. Response is returned to the frontend.
10. Dashboard displays updated information.

Integration Outcome

Milestone 1 successfully established communication between all major components of the application. The integrated system supports secure authentication, role-based dashboards, API communication, and database operations, providing a stable foundation for future AI-powered features.

5. Deliverables

The following deliverables were completed during Milestone 1:

Deliverable	Status
Project Statement	✅ Completed
Project Objectives	✅ Completed
Business Workflow	✅ Completed
User Roles	✅ Completed
Dataset Collection	✅ Completed
Dataset Analysis	✅ Completed
Data Cleaning & Preprocessing	✅ Completed
System Architecture	✅ Completed
Database Design	✅ Completed
UI Wireframes	✅ Completed
React Frontend	✅ Completed
FastAPI Backend	✅ Completed
JWT Authentication	✅ Completed
Role-Based Access Control	✅ Completed
API Development	✅ Completed
Dashboard Development	✅ Completed
GitHub Repository	✅ Completed
Project Documentation	✅ Completed

6. Milestone Outcomes

The successful completion of Milestone 1 resulted in the following outcomes:

- Established the overall project architecture.
- Designed the relational database schema.
- Developed a responsive React frontend.
- Implemented a modular FastAPI backend.
- Collected and processed retail datasets.
- Implemented secure JWT authentication.
- Configured role-based access control.
- Developed role-specific dashboards.
- Integrated frontend, backend, and database.
- Prepared the project for AI-based development in future milestones.

These achievements provide a strong technical foundation for implementing advanced analytics and machine learning features in the upcoming phases of the project.

---

7. Evaluation Criteria Mapping

The following table demonstrates how the completed work aligns with the internship evaluation criteria.

Evaluation Criteria	Evidence
Project Planning	Project Statement, Objectives, Business Workflow
Dataset Preparation	Dataset Collection, Analysis, Preprocessing
System Design	System Architecture, Database Design
Frontend Development	React.js, Dashboard UI, Role-Based Navigation
Backend Development	FastAPI, REST APIs, Database Integration
Authentication	JWT Authentication
Authorization	Role-Based Access Control
Documentation	Milestone Report

Evaluation Criteria	Evidence
Collaboration	GitHub Branch Workflow
Integration	Frontend–Backend–Database Integration

---

8. Challenges Faced

During the development of Milestone 1, the team encountered several challenges while designing and implementing the application.

Challenge	**Solution**
Dataset cleaning	Preprocessing pipeline
RBAC	JWT permissions
API integration	REST APIs
Merge conflicts	Git branches

---

9. Conclusion

Milestone 1 successfully established the technical and functional foundation of the MarketMind AI platform. The team completed dataset collection and preprocessing, designed the system architecture and database, developed the frontend and backend, implemented JWT authentication and Role-Based Access Control, and integrated the major application components.

The completed work provides a scalable and maintainable platform for implementing advanced business intelligence and AI capabilities in future milestones. By following a modular architecture and collaborative development practices, the team ensured that the application can be extended efficiently while maintaining code quality and security.

---

10. Milestone 2 Plan

The next phase of the project will focus on expanding the functionality of MarketMind AI by introducing advanced business intelligence and AI-driven features.

Planned activities include:

- Development of AI-powered sales forecasting.
- Customer segmentation using machine learning.

- Product recommendation system.
- Inventory demand prediction.
- Advanced analytics dashboard.
- Interactive business reports.
- Report export (PDF and Excel).
- Real-time notifications.
- Performance optimization.
- Comprehensive application testing and deployment.
